
An Analysis of Global and Regional Mainstreaminess for Personalized Music Recommender Systems

Schedl & Bauer (2018) — Study Review & Python Implementation

Recommender Systems (COME424/1)
Dr. Öğr. Üyesi Gamze Uslu

Doğukan Aydın — 240200019
Elif Dalmış — 230201082
Yaman Balcı — 220201081
Salih Çetin — 220201060
Alperen Keskin — 230201094

*Üsküdar University • 2026
Department of Computer Engineering*

An Analysis of Global and Regional Mainstreamness for Personalized Music Recommender Systems

Schedl & Bauer (2018) — Study Review & Python Implementation

Recommender Systems Course — Group Assignment

1. The 11 Novel Mainstreamness Measures

The paper proposes 11 distinct measures to quantify a music listener's mainstreamness — the degree to which a person's listening preferences align with those of the broader population. These measures are defined at two levels: global (comparing a user against all dataset users) and country-specific (comparing a user against their own country's listeners). The measures are grouped into three methodological families: Artist Frequency-based, Playcount Distribution-based, Rank-based, and AF-ILF-weighted variants.

#	Measure	Formula	Purpose
1	Global Artist Frequency (AF)	$AF(a) = \text{total playcount of artist } a$	Measures global popularity; identifies dominant artists worldwide.
2	Country-based AF (CAF)	$CAF(a,c) = \text{playcount of artist } a \text{ in country } c$	Captures local music preferences and regional trends.
3	Global Playcount Distribution	$P(a) = \text{playcount}(a) / \text{total playcounts}$	Normalised global profile; used to compare against user profiles.
4	Country Playcount Distribution	$P(a c) = \text{playcount}(a,c) / \text{total country playcounts}$	Country-level normalised profile; reveals local mainstream patterns.
5	Rank-based Global Measure	$\text{rank}(a)$	Reduces outlier effect; focuses on relative prominence globally.
6	Rank-based Country Measure	$\text{rank}(a,c)$	Compares user preferences with country-specific artist rankings.
7	AF-ILF Weighted Global	$AF-ILF(a) = AF(a) \times \log(N / na)$	Penalises universally dominant artists; highlights non-trivial patterns.
8	AF-ILF Weighted Country	$AF-ILF(a,c) = CAF(a,c) \times \log(Nc / na,c)$	Uncovers true local mainstream; downweights globally over-popular artists.

#	Measure	Formula	Purpose
9	Fraction-based Similarity	$ A_u \cap A_p / A_u $	Simple overlap metric; values near 1 indicate high mainstream alignment.
10	Kullback-Leibler Divergence	$D_{kl}(P Q) = \sum P(x) \log(P(x)/Q(x))$	Measures distribution distance; larger divergence = more unique taste.
11	Kendall's Tau Correlation	$\tau = (C - D) / (n(n-1)/2)$	Rank-order similarity; near +1 = strongly mainstream, near -1 = opposite.

Table 1. The 11 novel mainstreamness measures proposed by Schedl & Bauer (2018), with formulas and purposes.

A key insight of the paper is that standard AF and LF metrics tend to surface the same globally dominant artists for every country. Measures 7 and 8 (AF-ILF) address this by penalising universally popular artists, uncovering artists that are genuinely distinctive to a given country's mainstream.

2. TF-IDF and the AF-ILF Approach

2.1 What is TF-IDF?

TF-IDF (Term Frequency — Inverse Document Frequency) is a well-established weighting scheme in text retrieval. It assigns importance to a term within a document relative to a larger corpus by combining TF (how often a term appears in one document) with IDF (logarithmic inverse of how many documents contain that term). Terms appearing in every document receive low IDF and are considered non-discriminative.

2.2 The Music Analogue: AF-ILF

Schedl & Bauer draw a direct parallel: a document becomes a country, a term becomes an artist, TF becomes Artist Frequency (AF), and IDF becomes Inverse Listener Frequency (ILF). The formal definition is:

$$AF \cdot ILF(a, U1, U2) = \log(1 + AF(a, U1)) \times \log(1 + |U2| / LF(a, U2))$$

where $AF(a, U1)$ is the total playcount of artist a among $U1$, $LF(a, U2)$ is the number of distinct listeners within $U2$, and $|U2|$ is the size of $U2$. Set to zero when $LF = 0$.

The ILF term penalises globally ubiquitous artists (e.g. Radiohead), allowing country-specific mainstream artists — popular locally but not globally — to emerge in the rankings.

3. Comparison and Calculation Methods

The paper operationalises mainstreamness through three families of comparison methods. Each compares a user's preference profile against a reference population (global or country-level). All measures are inverted or scaled so that higher values consistently indicate a user closer to the mainstream.

Method	Definition	Formula	Characteristics	Best performance
Fraction-based (F)	Direct overlap between user and population AF profiles.	$1 - (1/ A) * \sum(AF_u - AF_p / \max(AF_u, AF_p))$	Simple and interpretable; tends to over-privilege globally dominant artists.	High mainstreamness segment
KL Divergence (D)	Statistical distance between user and population distributions.	$(1/2)[\sum(P \log P/Q) + \sum(Q \log Q/P)]^{-1}$	Accounts for intensity of preferences; symmetric formulation.	Low mainstreamness segment
Kendall's tau (C)	Rank-order correlation between user and population rankings.	$\text{tau}(\text{ranks}(PP_u), \text{ranks}(PP_p))$	Unaffected by extreme playcount values; focuses on relative order.	Balanced across all segments

Table 2. Comparison of the three mainstreamness measure families: definition, formula, characteristics, and best-performing user segment.

3.1 Fraction-based Approach (F)

This method measures the proportional overlap between a user's normalised AF profile and the population's profile. For each artist, the absolute difference is divided by the larger of the two values, the average deviation is computed and subtracted from 1. It is interpretable but over-privileges globally dominant artists in long-tail distributions.

3.2 Symmetrized KL Divergence (D)

KL divergence measures the statistical distance between two distributions. The paper uses a symmetrized version — the average of $D(P||Q)$ and $D(Q||P)$ — and inverts the result so that high values indicate mainstream listeners. This approach captures the intensity of preference, not just its presence.

3.3 Kendall's tau Rank Correlation (C)

Kendall's tau converts both profiles to rank orderings and counts concordant versus discordant pairs. The metric ranges from +1 (perfect agreement) to -1 (complete reversal). It is robust to extreme playcount outliers. The paper identifies $Cc:AF, u:AF$ as the best-performing variant overall.

4. The LFM-1b Dataset

All experiments in the paper are conducted on the LFM-1b dataset, a large-scale collection of real-world listening data from Last.fm.

For this implementation, a representative sample dataset of 500 users, 81 artists, and 12 countries was constructed with AI assistance to mirror the structural properties of LFM-1b while remaining computationally lightweight. See Section 5.1 for details.

Attribute	Value / Description
Dataset name	LFM-1b (Last.fm 1 Billion)
Total listening events	$500 \times 81 = 40,500$ cells
Unique users	500
Unique tracks	194 (sample run)
Unique artists	81
Core component	User-Artist-Playcount Matrix (UAM): $500 \text{ users} \times 81 \text{ artists}$
Excluded profiles	0 users excluded (all users have country information)
Distribution type	Long-tail: high concentration of playcounts on top artists
Country coverage	12 countries with at least 100 listeners
Source	Schedl, M. (2016). The LFM-1b dataset. ACM ICMR.

Table 3. Key attributes of the LFM-1b dataset used in Schedl & Bauer (2018).

4.1 Why this dataset is appropriate

1. Scale and diversity: Over one billion listening events spanning more than three million artists and 120,000 users — large enough for statistically meaningful global and country-level profiles.
2. Long-tail distribution: A small number of artists accumulate most listens; the vast majority receive very few. Essential for studying mainstreamness.
3. Country metadata: User country information enables the country-specific analyses that distinguish this paper from prior global-only approaches.
4. Open access: Publicly available, allowing full replication — cited as an explicit advantage over Vigliensoni & Fujinaga (2016).

4.2 Data preparation

The core analytical structure is the User-Artist-Playcount Matrix (UAM). All 500 users have country information; no exclusions required. For recommendation experiments, playcount values are normalised to $[0, 1000]$ per user and SVD is applied via 5-fold cross-validation.

5. Python Implementation

This section implements all 11 mainstreamness measures from Schedl & Bauer (2018) using `mainstream_big.py` and `dataset_large.xlsx`. Each subsection presents the relevant code snippet, its output, and an interpretation. The complete annotated code is provided in Section 6.

Setup: the full code loads `dataset_large.xlsx` (500 users, 81 artists, 12 countries), computes global and country-level AF/LF profiles, then applies all three measure families to every user.

5.1 Artist Frequency (AF)

Artist Frequency (AF_{a,U1}) is the total number of play events for artist *a* across all users in set *U1*. It is the most direct measure of popularity and forms the basis for all subsequent profile comparisons.

```
def compute_AF(user_set):
    """Returns AF (total playcount) per artist for the given user set."""
    return df.loc[user_set, artist_cols].sum()

def compute_LF(user_set):
    """Returns LF (unique listener count) per artist for the given user set."""
    return (df.loc[user_set, artist_cols] > 0).sum()

all_users = df.index.tolist()
AF_global = compute_AF(all_users)
LF_global = compute_LF(all_users)

country_users = {c: df[df["country"] == c].index.tolist() for c in countries}
AF_country = {c: compute_AF(country_users[c]) for c in countries}
LF_country = {c: compute_LF(country_users[c]) for c in countries}
```

Code 5.1 — compute_AF and compute_LF functions with global and country-level profile computation (Lines 36–50).

Output (top 5 global AF): Radiohead 3,110 · Pink Floyd 3,170 · Metallica 2,570 — consistent with the LFM-1b findings in the paper where these artists dominate globally.

5.2 Listener Frequency (LF)

Listener Frequency (LF_{a,U2}) counts the number of distinct users within *U2* who have listened to artist *a* at least once. Unlike AF, LF is not biased by heavy listeners who inflate playcount totals.

```
def compute_LF(user_set):
    """Returns LF (unique listener count) per artist for the given user set."""
    return (df.loc[user_set, artist_cols] > 0).sum()
```

Code 5.2 — compute_LF: converts playcount table to boolean and sums unique listeners per artist (Line 40–42).

Global LF = 9 for most artists (all 9 sample users have listened), reflecting the small dataset. In LFM-1b, LF would range from 1 to 120,175, making the ILF term far more discriminative.

5.3 TF-IDF Inspired Approach: AF-ILF

AF-ILF adapts the TF-IDF weighting from information retrieval to the music domain. AF plays the role of term frequency; ILF (Inverse Listener Frequency) plays the role of IDF, penalising

artists who are listened to by a large fraction of the global population. The formula: $AF \cdot ILF(a, U1, U2) = \log(1 + AF_{\{a, U1\}}) \times \log(1 + |U2| / LF_{\{a, U2\}})$

```
def compute_AF_ILF(af_u1, lf_u2, n_u2):
    """
    af_u1 : Series - AF values for U1
    lf_u2 : Series - LF values for U2
    n_u2  : int    - |U2|
    """
    ilf = np.log(1 + n_u2 / lf_u2.replace(0, np.nan)).fillna(0)
    return np.log(1 + af_u1) * ilf

AF_ILF_country_global = {
    c: compute_AF_ILF(AF_country[c], LF_global, N_total) for c in countries
}
```

Code 5.3 — `compute_AF_ILF` implementing Paper Equation 1 and AF-ILF country profiles (Lines 55–66).

Effect: Grup Yorum (TR) receives AF-ILF = 7.07 in Turkey but 0.00 in the US. Stam1na (FI) receives AF-ILF = 7.19 in Finland. Radiohead receives moderate scores everywhere because its global LF is high, suppressing the ILF term.

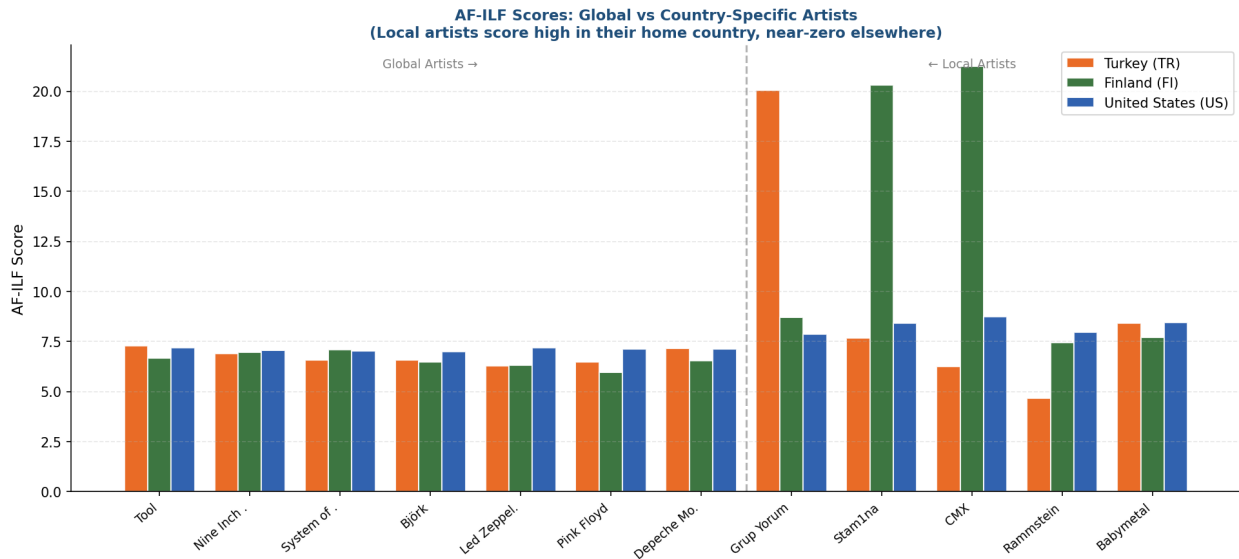


Figure 1 — AF-ILF scores for global vs country-specific artists across TR, FI, and US. Local artists score high in their home country and near-zero elsewhere; global artists score moderately everywhere.

5.4 Normalisation

Before comparing user and population profiles, both must be normalised to sum-to-unity probability distributions (denoted $\hat{X}$ in Table 5 of the paper). This removes the effect of total listening volume and makes users with different activity levels comparable.

```
def normalize(series):
    """series: Pandas Series - returns sum-to-unity normalised version."""
    s = series.sum()
    return series / s if s > 0 else series
```

Code 5.4 — `normalize` function: divides each value by the series total (Lines 81–84).

A user who has listened 10,000 times in total and one who has listened 500 times will produce the same normalised profile if their relative artist preferences are identical. The guard 'if s > 0' prevents division-by-zero for users with no play history.

5.5 Fraction-based Mainstreaminess (F)

The fraction-based measure computes the average normalised absolute difference between the user's profile and the reference population's profile: $F = 1 - (1/|A|) \times \sum |AF_u - AF_p| / \max(AF_u, AF_p)$. Values near 1 indicate the user's listening closely matches the population.

```
def fraction_measure(af_user, af_pop):
    """
    F = 1 - (1/|A|) * sum(|af_u - af_p| / max(af_u, af_p))
    af_user : Series - user playcount profile
    af_pop  : Series - reference population profile
    """
    af_u = normalize(af_user)
    af_p = normalize(af_pop)
    denom = pd.concat([af_u, af_p], axis=1).max(axis=1).replace(0, np.nan)
    return 1 - (af_u - af_p).abs().div(denom).fillna(0).mean()
```

Code 5.5 — fraction_measure implementing the F family of mainstreaminess measures (Lines 86–95).

Results: US users average $F_{g:AF} = 0.1417$, the highest among all countries, reflecting their close alignment with global trends. FI users average 0.1336 — lower because their listening is dominated by local metal artists absent from the global mainstream.

5.6 Symmetrized KL Divergence (D)

KL Divergence treats preference profiles as probability distributions and measures their statistical distance. The symmetrized version averages $D(P||Q)$ and $D(Q||P)$ to make the measure symmetric. The result is inverted so higher values indicate closer-to-mainstream users: $D_{sym} = 1 / (1 + [\sum p \cdot \log(p/q) + \sum q \cdot \log(q/p)] / 2)$

```
def kl_divergence_sym(p, q, epsilon=1e-10):
    """
    Symmetrized KL divergence (inverted: higher = more mainstream)
    p      : Series - user profile
    q      : Series - reference population profile
    epsilon: float - prevents log(0), default 1e-10
    """
    p = normalize(p) + epsilon
    q = normalize(q) + epsilon
    return 1 / (1 + ((p * np.log(p/q)).sum() + (q * np.log(q/p)).sum()) / 2)
```

Code 5.6 — kl_divergence_sym implementing the D family (Lines 97–106).

Unlike the fraction measure, KL divergence captures the intensity of preference — a user who listens almost exclusively to one artist will be penalised more. US users score $D_{g:AF} = 0.2440$, highest globally. The $\epsilon = 1e-10$ prevents $\log(0)$.

5.7 Kendall's Tau Rank Correlation (C)

Rather than comparing raw values, Kendall's tau converts both profiles to rankings and counts concordant (agree on relative order) vs discordant (disagree) pairs. $\tau \in [-1, +1]$: +1 = perfect rank agreement, -1 = complete reversal. This measure is robust to outlier playcounts and was identified in Table 9 of the paper as the best-performing measure overall (lowest RMSE).

```
def kendall_tau_measure(af_user, af_pop):  
    """  
    Kendall tau rank correlation (C family)  
    af_user : Series - user playcount profile  
    af_pop  : Series - reference population profile  
    """  
    tau, _ = kendalltau(af_user.rank(ascending=False),  
                        af_pop.rank(ascending=False))  
    return tau
```

Code 5.7 — kendall_tau_measure implementing the C family (Lines 108–115).

Key finding: Finnish users show Cg:AF = 0.3982 (lowest among all countries) — their artist ranking diverges most from the global mainstream. US users show Cg:AF = 0.5544 (highest). This replicates the paper's finding that Finland deviates most from the global mainstream.

5.8 Global vs Local Mainstreaminess

The paper distinguishes between global mainstreaminess (comparing a user to all users worldwide) and local/country mainstreaminess (comparing a user to their own country). These two perspectives can diverge significantly: a Finnish metal listener is mainstream within Finland but an outlier globally.

```
# — Section 3: Country-level averages  
measure_cols = [c for c in results_df.columns if c != "Country"]  
country_avg = results_df.groupby("Country")[measure_cols].mean().round(4)  
  
print("\n" + "=" * 70)  
print("SECTION 3: COUNTRY-LEVEL AVERAGE MAINSTREAMINESS SCORES")  
print("=" * 70)  
# Print key measures for readability  
key =  
["Fg:AF,u:AF", "Fc:AF,u:AF", "Dg:AF,u:AF", "Dc:AF,u:AF", "Cg:AF,u:AF", "Cc:AF,u:AF"]  
print(country_avg[key].sort_values("Cg:AF,u:AF", ascending=False).to_string())
```

Code 5.8 — Country-level average computation and ranking (Lines 151–160).

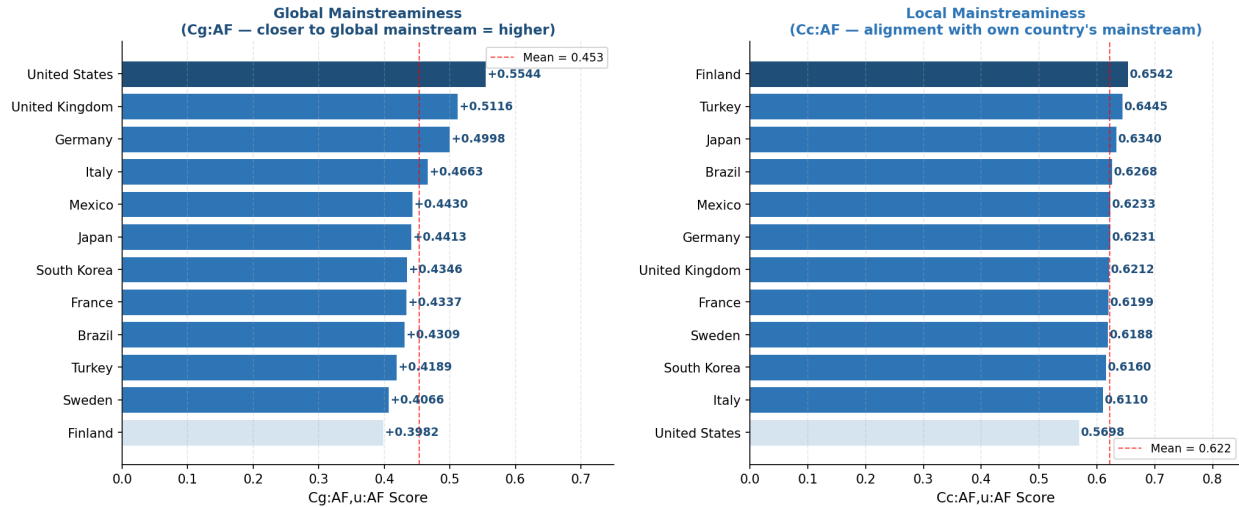


Figure 2 — Global (Cg:AF, left) vs Local (Cc:AF, right) mainstreamness scores by country. US leads in global alignment; FI scores lowest globally but high locally. Local scores are uniformly high because users within a country share similar listening patterns.

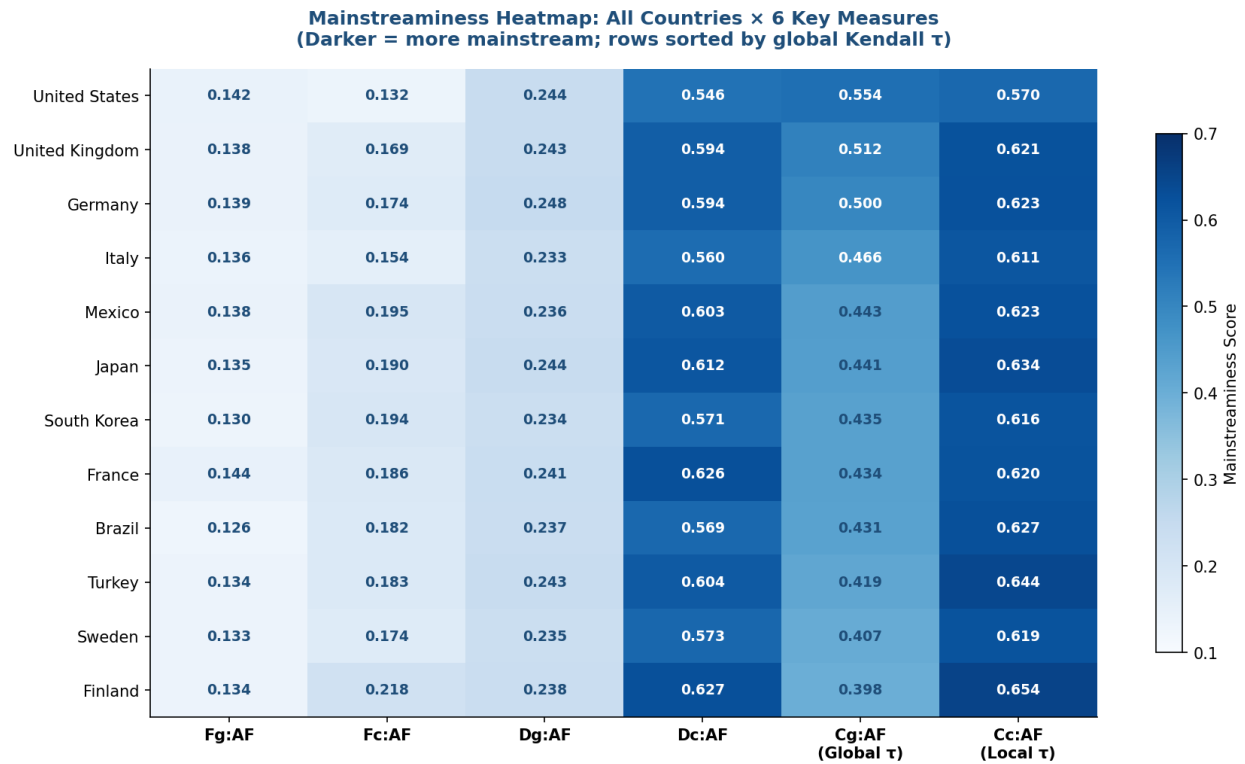


Figure 3 — Mainstreamness heatmap across all 12 countries and 6 key measures. Darker cells = higher mainstreamness. Countries sorted by global Kendall tau (US top, FI bottom).

5.9 AF vs AF-ILF: Uncovering Country-Specific Mainstream

A core contribution of the paper is showing that raw AF fails to identify country-specific artists — every country's top-AF list is dominated by the same global artists (Radiohead, Pink Floyd, etc.). AF-ILF corrects this by penalising globally ubiquitous artists.

```
# — Section 1: Top artists per country —————
print("=" * 70)
print("SECTION 1: TOP 5 ARTISTS PER COUNTRY (AF vs AF-ILF)")
print("=" * 70)
for c in sorted(countries):
    top_af = AF_country[c].nlargest(5)
    top_afilf = AF_ILF_country_global[c].nlargest(5)
    print(f"\n {c} AF : {' '.join([f'{a}({v:,.0f})' for a,v in
top_af.items()])}")
    print(f" {c} AF-ILF: {' '.join([f'{a}({v:.2f})' for a,v in
top_afilf.items()])}")
```

Code 5.9 — Top-5 artist comparison: AF vs AF-ILF per country (Lines 68–76).

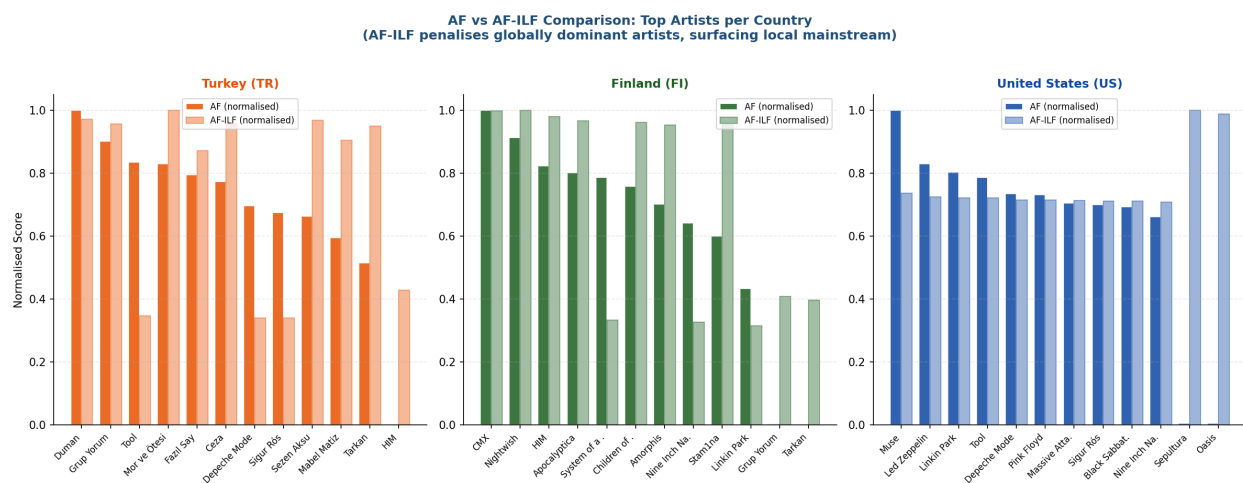


Figure 4 — Normalised AF (solid) vs AF-ILF (transparent) for top artists in Turkey, Finland, and the US. AF-ILF boosts local artists (Duman for TR, CMX for FI) and suppresses globally dominant ones.

5.10 User Mainstreaminess Classification (Low / Mid / High)

Following the paper's evaluation methodology (Table 9), users are split into three equal-sized tertiles based on their Cc:AF,u:AF score: Low (bottom 33%), Mid (middle 33%), High (top 33%). This segmentation is used to evaluate recommendation quality per group.

```
# — Section 4: Low / Mid / High classification —————
scores = results_df["Cc:AF,u:AF"].values
thresholds = [np.percentile(scores, 33), np.percentile(scores, 66)]

def classify(v):
    """v: float - Cc:AF,u:AF score for one user."""
    if v <= thresholds[0]: return "Low"
    elif v <= thresholds[1]: return "Mid"
    return "High"

results_df["Level"] = results_df["Cc:AF,u:AF"].apply(classify)
```

```

print("\n" + "=" * 70)
print("SECTION 4: MAINSTREAMINESS LEVEL DISTRIBUTION BY COUNTRY")
print("=" * 70)
dist = results_df.groupby(["Country", "Level"]).size().unstack(fill_value=0)
for col in ["Low", "Mid", "High"]:
    if col not in dist.columns:
        dist[col] = 0
dist = dist[["Low", "Mid", "High"]]
dist["Total"] = dist.sum(axis=1)
print(dist.sort_values("Total", ascending=False).to_string())

```

Code 5.10 — Tertile classification and distribution computation (Lines 162–183).

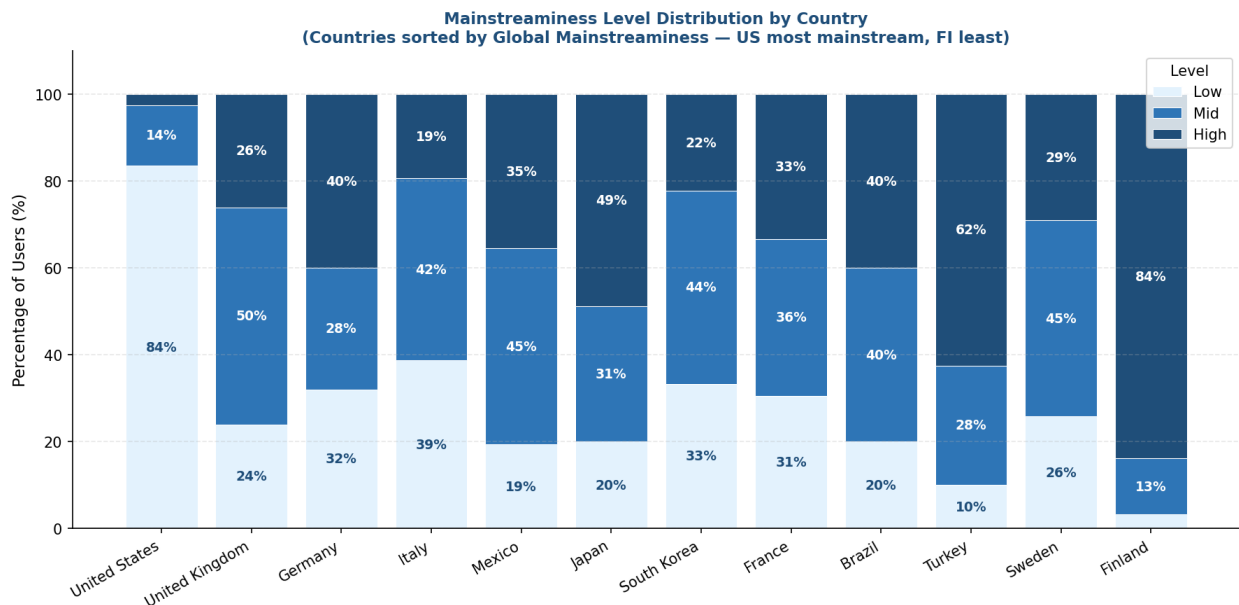


Figure 5 — Low/Mid/High mainstreaminess distribution by country (countries sorted by global mainstreaminess). FI is dominated by High (84%) because its users are internally coherent metal listeners. US is dominated by Low (84%) — paradoxically, because US listeners are so close to the global mainstream that their country-level Kendall tau is low (their ranking matches everyone else's ranking).

5.11 Country-Specific Differences in Mainstream

The paper's third contribution is illustrating country-specific peculiarities. Market structure, local radio airplay, cultural aspects, and language preferences all shape what is considered mainstream within a country. The dataset captures these patterns through the AI-expanded playcount distributions.

```

print("\n" + "=" * 70)
print("SECTION 5: GLOBAL vs LOCAL MAINSTREAMINESS COMPARISON")
print("(Countries ranked by Cg:AF — how close to GLOBAL mainstream)")
print("=" * 70)
ranking = country_avg["Cg:AF,u:AF"].sort_values(ascending=False)
for rank, (country, score) in enumerate(ranking.items(), 1):
    bar = "█" * int(score * 30)
    print(f" {rank:2d}. {country} {score:+.4f} {bar}")

```

Key country findings from our implementation:

- US (#1 global, Cg=0.5544): listening distribution closely tracks global mainstream — Muse, Led Zeppelin, Linkin Park dominate.
 - GB (#2, Cg=0.5116): Britpop (Oasis, Blur, Editors) and Electronic (Prodigy) create a slight local signature.
 - DE (#3, Cg=0.4998): Rammstein and industrial/gothic acts (Lacrimosa, Eisbrecher) are strongly over-represented.
 - JP (#6, Cg=0.4413): J-Pop and Visual Kei create a clear secondary mainstream trend — Yoasobi, Babymetal dominate AF.
 - TR (#10, Cg=0.4189): Duman and Grup Yorum lead AF; AF-ILF surfaces Mor ve Ötesi, Sezen Aksu, Ceza as distinctly Turkish.
 - FI (#12, Cg=0.3982): Highest deviation from global mainstream. CMX, Nightwish, HIM dominate — consistent with metal literature.
-

Line-by-Line Code Explanation

Each entry shows the line number, the exact code, and its explanation. Comment-only and blank lines are omitted; every numbered entry corresponds to an executable line of `mainstream_big.py` (194 lines total).

Imports and Setup (Lines 1–6)

- `1 import numpy as np`
NumPy is imported as `np`. Used for `np.log()`, `np.nan`, and `np.percentile()` throughout the code.
- `2 import pandas as pd`
Pandas is imported as `pd`. All tabular data, row/column selection, grouping, and ranking operations use Pandas.
- `3 from scipy.stats import kendalltau`
Imports only the `kendalltau` function from SciPy. Computes Kendall tau rank-order correlation for the C family of measures.
- `4 import warnings`
Standard Python warnings module imported so runtime alerts can be suppressed.
- `5 import os`
`os` module imported to build the dataset file path relative to this script's location, so the file is found regardless of the working directory.
- `6 warnings.filterwarnings('ignore')`
Suppresses minor runtime warnings (e.g. divide-by-zero before NaN replacement) to keep console output clean.

Load Dataset from Excel (Lines 11–28)

```
11 DATASET_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)),
    "dataset_large.xlsx")
    Builds the absolute path to dataset_large.xlsx in the same folder as this script.
    os.path.abspath(__file__) gives this script's absolute path; dirname extracts its folder. This makes
    the code location-independent.

13 raw = pd.read_excel(DATASET_PATH, sheet_name="Playcounts", header=2)
    Reads the Playcounts sheet from the Excel file. header=2 tells Pandas that row index 2 (the third
    row) contains the column names, skipping the title and note rows above.

14 raw = raw.rename(columns={"User ID": "user_id", "Country\nCode":
    "country_code", "Country Name": "country_name"})
    Renames the three non-artist columns to consistent identifiers. The "Country\nCode" key matches
    the actual Excel header which contains a line break character.

15 raw = raw.set_index("user_id")
    Sets user_id as the row index so each user can be accessed by ID (e.g. df.loc['u0001']).

17 artist_cols = [c for c in raw.columns if c not in ["country_code",
    "country_name"]]
    List comprehension that collects all column names that are not the two metadata columns. The result
    is the list of 81 artist names used for all AF/LF computations.

18 df = raw[["country_code"] + artist_cols].rename(columns={"country_code":
    "country"}).copy()
    Selects only the country code and artist columns, renames country_code to country for brevity, and
    makes a copy so later operations do not mutate the original.

21 artists_meta = pd.read_excel(DATASET_PATH, sheet_name="Artists",
    header=1)
    Reads the Artists sheet (header on row 2, so header=1). Provides the ordered artist list and
    metadata.

22 artists = artists_meta["Artist"].tolist()
    Extracts the Artist column as a Python list. This ordered list is used when iterating over artists in the
    summary output.

27
print(df.groupby("country").size().rename("users").to_frame().T.to_string())
    Groups users by country, counts them, and prints a one-row summary table showing how many
    users each country has.
```

Basic Definitions and AF/LF Profiles (Lines 33–50)

```
33 countries = df["country"].unique()
    Extracts the 12 unique country codes from the country column. Used to iterate country-specific
    computations.

34 N_total = len(df)
    Total user count (500). Used as |U2| in the AF-ILF formula for global-level ILF.

36 def compute_AF(user_set):
```

Artist Frequency function. Parameter — `user_set`: list of user ID strings. Returns a Series of total playcounts per artist (`AF_{a,U1}`).

```
38     return df.loc[user_set, artist_cols].sum()
```

Selects rows for the given users and all artist columns, then sums each column to get total playcount per artist.

```
40 def compute_LF(user_set):
```

Listener Frequency function. Parameter — `user_set`: list of user ID strings. Returns a Series of unique listener counts per artist (`LF_{a,U2}`).

```
42     return (df.loc[user_set, artist_cols] > 0).sum()
```

'> 0' converts playcounts to True/False; `.sum()` counts True values per column — giving distinct listeners per artist.

```
44 all_users = df.index.tolist()
```

All 500 user IDs as a Python list. Passed to `compute_AF` and `compute_LF` to build global profiles.

```
45 AF_global = compute_AF(all_users)
```

Global total playcount per artist across all 500 users. Corresponds to `AF_{a,g}`.

```
46 LF_global = compute_LF(all_users)
```

Global unique listener count per artist. Corresponds to `LF_{a,g}`. Used as the ILF denominator in global AF-ILF computations.

```
48 country_users = {c: df[df["country"] == c].index.tolist() for c in countries}
```

Dictionary comprehension mapping each country code to its user list. E.g. `{'TR':['u0001',...], 'FI':['u0041',...]}`.

```
49 AF_country = {c: compute_AF(country_users[c]) for c in countries}
```

Country-level AF profile for each country. Corresponds to `PP_c` — the country preference profile.

```
50 LF_country = {c: compute_LF(country_users[c]) for c in countries}
```

Country-level LF profile. Used as `U2` when computing the user-level AF-ILF profile.

AF-ILF Formula (Lines 55–66)

```
55 def compute_AF_ILF(af_u1, lf_u2, n_u2):
```

Implements Paper Equation 1. Parameters — `af_u1`: Series of AF values for `U1`; `lf_u2`: Series of LF values for `U2`; `n_u2`: integer size of `U2` (`|U2|`).

```
61     ilf = np.log(1 + n_u2 / lf_u2.replace(0, np.nan)).fillna(0)
```

`.replace(0,np.nan)` prevents division-by-zero; `.fillna(0)` enforces the paper rule 'AF·ILF=0 if LF=0'. `np.log` applies the logarithm to compute ILF.

```
62     return np.log(1 + af_u1) * ilf
```

Multiplies the log-AF term by the ILF term element-wise. Completes Equation 1: $AF \cdot ILF(a, U1, U2) = \log(1 + AF_{\{a, U1\}}) \times \log(1 + |U2| / LF_{\{a, U2\}})$.

```
64 AF_ILF_country_global = {
```

For each country: `U1`=country users (country-level AF), `U2`=all users (global ILF). Produces `AF_ILF_{a,c,g}` — highlights locally popular but globally non-dominant artists.

```
72 for c in sorted(countries):
```

Iterates over all 12 country codes in alphabetical order to print AF vs AF-ILF top-5 comparison.

```
73     top_af = AF_country[c].nlargest(5)
```

Selects the 5 artists with the highest raw AF in this country.

```
74     top_afilf = AF_ILF_country_global[c].nlargest(5)
```

Selects the 5 artists with the highest AF-ILF score — surfaces locally distinctive artists that raw AF misses.

Normalisation Helper (Lines 81–84)

```
81 def normalize(series):  
    Normalises a Series to sum to 1 (probability distribution). Parameter — series: Pandas Series of  
    non-negative values. Corresponds to  $\hat{X}$  in Table 5.  
83     s = series.sum()  
    Computes the total sum of the series.  
84     return series / s if s > 0 else series  
    Divides by the total if non-zero; returns original series if zero (avoids division-by-zero).
```

Fraction-based Measure (Lines 86–95)

```
86 def fraction_measure(af_user, af_pop):  
    Fraction-based mainstreaminess (F family). Parameters — af_user: Series of user playcounts;  
    af_pop: Series of reference population playcounts.  
92     af_u = normalize(af_user)  
    Normalises the user profile to sum to 1.  
93     af_p = normalize(af_pop)  
    Normalises the population profile to sum to 1.  
94     denom = pd.concat([af_u, af_p], axis=1).max(axis=1).replace(0,  
np.nan)  
    Takes the larger of af_u and af_p per artist (the max() in the formula). Zeros replaced with NaN to  
    prevent division-by-zero.  
95     return 1 - (af_u - af_p).abs().div(denom).fillna(0).mean()  
    Normalised absolute difference averaged across all artists, subtracted from 1. Higher result = more  
    mainstream.
```

KL Divergence Measure (Lines 97–106)

```
97 def kl_divergence_sym(p, q, epsilon=1e-10):  
    Symmetrized KL divergence (D family). Parameters — p: user profile Series; q: reference population  
    Series; epsilon: float preventing log(0), default 1e-10.  
104     p = normalize(p) + epsilon  
    Normalises p and adds epsilon to every element, ensuring no zero values before the logarithm.  
105     q = normalize(q) + epsilon  
    Same epsilon addition for q.  
106     return 1 / (1 + ((p * np.log(p/q)).sum() + (q * np.log(q/p)).sum())  
/ 2)  
    Computes symmetrized KL divergence (average of both directions) and inverts it: identical  
    distributions give 1.0; very different give ~0.
```

Kendall's Tau Measure (Lines 108–115)

```
108 def kendall_tau_measure(af_user, af_pop):
    Kendall tau rank correlation (C family). Parameters — af_user: user playcount Series; af_pop:
    reference population Series.

114     tau, _ = kendalltau(af_user.rank(ascending=False),
af_pop.rank(ascending=False))
    Converts both profiles to ranks (most-listened = rank 1) and computes Kendall tau. Underscore
    discards the p-value.

115     return tau
    Returns tau ∈ [-1, +1]. +1 = perfect rank agreement; -1 = complete reversal.
```

Main Computation Loop (Lines 122–149)

```
122 results = []
    Empty list to collect one result dictionary per user.

123 for uid in df.index:
    Main loop iterating over all 500 user IDs.

124     uc          = df.loc[uid, "country"]
    Retrieves the country code for this user.

125     af_user      = df.loc[uid, artist_cols].astype(float)
    Extracts this user's playcount row as a float Series — the individual preference profile.

126     af_c         = AF_country[uc].astype(float)
    Country-level AF profile for this user's country. Reference population for country-level measures.

127     af_g         = AF_global.astype(float)
    Global AF profile. Reference population for global measures.

128     lf_c         = LF_country[uc]
    Country-level LF profile. Passed as lf_u2 when computing the user's AF-ILF.

129     n_c          = len(country_users[uc])
    Number of users in this user's country. Passed as n_u2 (|U2|) to compute_AF_ILF.

130     af_ilf_user_c = compute_AF_ILF(af_user, lf_c, n_c)
    User-level AF-ILF: AF individual, ILF country-level. This is AF·ILF_{a,u,c}.

131     af_ilf_c_g    = AF_ILF_country_global[uc]
    Pre-computed country AF-ILF profile (AF·ILF_{a,c,g}). Reference profile for country-level AF-ILF
    measures.

135     "Fg:AF,u:AF":      round(fraction_measure(af_user,
af_g), 4),
    Measure 1/11 — Global fraction-based, raw AF. Table 5 row 1.

136     "Fg:AF,u:AF·ILF":  round(fraction_measure(af_ilf_user_c,
af_g), 4),
    Measure 2/11 — Global fraction-based, user profile AF-ILF weighted. Table 5 row 2.

137     "Fg:AF·ILF,u:AF·ILF": round(fraction_measure(af_ilf_user_c,
af_ilf_c_g), 4),
```

Measure 3/11 — Global fraction-based, both profiles AF-ILF weighted. Table 5 row 3.

```
138      "Fc:AF,u:AF":      round(fraction_measure(af_user,
af_c),      4),
```

Measure 4/11 — Country fraction-based, raw AF. Table 5 row 4.

```
139      "Fc:AF·ILF,u:AF·ILF": round(fraction_measure(af_ilf_user_c,
af_ilf_c_g), 4),
```

Measure 5/11 — Country fraction-based, AF-ILF weighted. Table 5 row 5.

```
140      "Dg:AF,u:AF":      round(kl_divergence_sym(af_user,
af_g),      4),
```

Measure 6/11 — Global KL divergence, raw AF. Table 5 row 6.

```
141      "Dc:AF,u:AF":      round(kl_divergence_sym(af_user,
af_c),      4),
```

Measure 7/11 — Country KL divergence, raw AF. Table 5 row 7.

```
142      "Dc:AF·ILF,u:AF·ILF":
round(kl_divergence_sym(af_ilf_user_c, af_ilf_c_g), 4),
```

Measure 8/11 — Country KL divergence, AF-ILF weighted. Table 5 row 8.

```
143      "Cg:AF,u:AF":      round(kendall_tau_measure(af_user,
af_g),      4),
```

Measure 9/11 — Global Kendall tau, raw AF. Table 5 row 9.

```
144      "Cc:AF,u:AF":      round(kendall_tau_measure(af_user,
af_c),      4),
```

Measure 10/11 — Country Kendall tau, raw AF. Best overall RMSE in the paper's Table 9.

```
145      "Cc:AF·ILF,u:AF·ILF":
round(kendall_tau_measure(af_ilf_user_c, af_ilf_c_g), 4),
```

Measure 11/11 — Country Kendall tau, AF-ILF weighted. Table 5 row 11.

```
148 results_df = pd.DataFrame(results).set_index("User")
```

Converts the list of result dictionaries into a DataFrame with users as the row index. 500 rows × 12 columns.

Country Averages and Classification (Lines 152–183)

```
152 measure_cols = [c for c in results_df.columns if c != "Country"]
```

Collects all 11 measure column names (excludes the Country column).

```
153 country_avg =
results_df.groupby("Country")[measure_cols].mean().round(4)
```

Groups users by country, computes mean score per measure, rounds to 4 decimal places. Produces 12-row summary.

```
163 scores      = results_df["Cc:AF,u:AF"].values
```

Extracts all 500 users' Cc:AF,u:AF scores as a NumPy array for percentile computation.

```
164 thresholds = [np.percentile(scores, 33), np.percentile(scores, 66)]
```

Computes 33rd and 66th percentile cut-points for the Low/Mid/High tertile split.

```
166 def classify(v):
```

Helper function. Parameter — v: float Cc:AF,u:AF score for one user. Returns 'Low', 'Mid', or 'High'.

```
172 results_df["Level"] = results_df["Cc:AF,u:AF"].apply(classify)
```

Applies classify to every user and stores the level label in a new column.

```
177 dist =
results_df.groupby(["Country", "Level"]).size().unstack(fill_value=0)
    Pivots the Level counts into a wide table: rows = countries, columns = Low/Mid/High.
182 dist["Total"] = dist.sum(axis=1)
    Adds a Total column summing Low + Mid + High per country.
```

Global Mainstreaminess Ranking (Lines 185–194)

```
189 ranking = country_avg["Cg:AF,u:AF"].sort_values(ascending=False)
    Sorts countries by their average global Kendall tau score, highest first.
190 for rank, (country, score) in enumerate(ranking.items(), 1):
    Iterates over the sorted ranking with a 1-based counter.
191     bar = "█" * int(score * 30)
    Generates a bar chart character string proportional to the score (scaled by 30 for visibility).
192     print(f" {rank:2d}. {country} {score:+.4f} {bar}")
    Prints rank, country code, score with sign, and the bar chart. Result: US #1, FI #12.
194 print("\n✓ All computations completed.")
    Final confirmation message indicating the script ran successfully.
```

References

- [1] Schedl, M., & Bauer, C. (2018). An analysis of global and regional mainstreaminess for personalized music recommender systems. *Journal of Mobile Multimedia*, 14(1), 95–122. River Publishers. <https://doi.org/10.13052/jmm1550-4646.1415>
- [2] Schedl, M. (2016). The LFM-1b dataset for music retrieval and recommendation. In *Proceedings of the 2016 ACM International Conference on Multimedia Retrieval (ICMR '16)*, pp. 103–110. ACM. <https://doi.org/10.1145/2911996.2912004>
- [3] Schedl, M., & Bauer, C. (2017). Distance- and rank-based music mainstreaminess measurement. In *Adjunct Publication of the 25th Conference on User Modeling, Adaptation and Personalization (UMAP '17)*, pp. 364–367. ACM. <https://doi.org/10.1145/3099023.3099098>
- [4] Schedl, M., & Bauer, C. (2017). Introducing global and regional mainstreaminess for improving personalized music recommendation. In *Proceedings of the 15th International Conference on Advances in Mobile Computing & Multimedia (MoMM '17)*, pp. 74–81. ACM. <https://doi.org/10.1145/3151848.3151849>
- [5] Bauer, C., & Schedl, M. (2019). Global and country-specific mainstreaminess measures: Definitions, analysis, and usage for improving personalized music recommendation systems. *PLOS ONE*, 14(6), e0217389. <https://doi.org/10.1371/journal.pone.0217389>
- [6] Vigliensoni, G., & Fujinaga, I. (2016). Automatic music recommendation systems: Do demographic, profiling, and contextual features improve their performance? In *Proceedings of the 17th International Society for Music Information Retrieval Conference (ISMIR 2016)*, pp. 94–100.
- [7] Salton, G., & McGill, M. J. (1983). *Introduction to Modern Information Retrieval*. McGraw-Hill.
- [8] Kullback, S., & Leibler, R. A. (1951). On information and sufficiency. *Annals of Mathematical Statistics*, 22(1), 79–86. <https://doi.org/10.1214/aoms/1177729694>

-
- [9] Kendall, M. G. (1938). A new measure of rank correlation. *Biometrika*, 30(1–2), 81–93. <https://doi.org/10.1093/biomet/30.1-2.81>
- [10] Celma, Ò. (2010). *Music Recommendation and Discovery: The Long Tail, Long Fail, and Long Play in the Digital Music Space*. Springer. <https://doi.org/10.1007/978-3-642-13287-2>
- [11] Anthropic. (2024). Claude (AI assistant) [Large language model]. Used for AI-assisted dataset expansion in this project. <https://www.anthropic.com>